EXP NO: 7

GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK

AIM:

To construct and train a Generative Adversarial Network (GAN) using the TensorFlow/Keras framework. The objective is to train the GAN on the MNIST dataset to generate new, synthetic images of handwritten digits that are indistinguishable from the original training data.

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os

tf.keras.utils.disable_interactive_logging()

print("TensorFlow version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))
```

```
TensorFlow version: 2.22.0-rc0
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA
Flow-DirectML plugin.
GPU: []
```

[2]:
```
(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1).astype("float32")
x_train = (x_train - 127.5) / 127.5

print("Training data shape:", x_train.shape)
print("Example normalized pixel:", x_train[0, 0, 0, 0])
```

```
Training data shape: (60000, 28, 28, 1)
Example normalized pixel: -1.0
```

[3]:
```
latent_dim = 100
batch_size = 256
EPOCHS = 200
num_examples_to_generate = 16
```

[4]:
```
EPOCHS = 20
```

[5]:
```
def build_generator():
    model = keras.Sequential(name="generator")

    model.add(layers.Input(shape=(latent_dim,)))
    model.add(layers.Dense(7 * 7 * 256, use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))

    model.add(layers.Conv2DTranspose(
        128, (5, 5), strides=(1, 1),
        padding="same", use_bias=False
    ))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(
        64, (5, 5), strides=(2, 2),
        padding="same", use_bias=False
    ))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

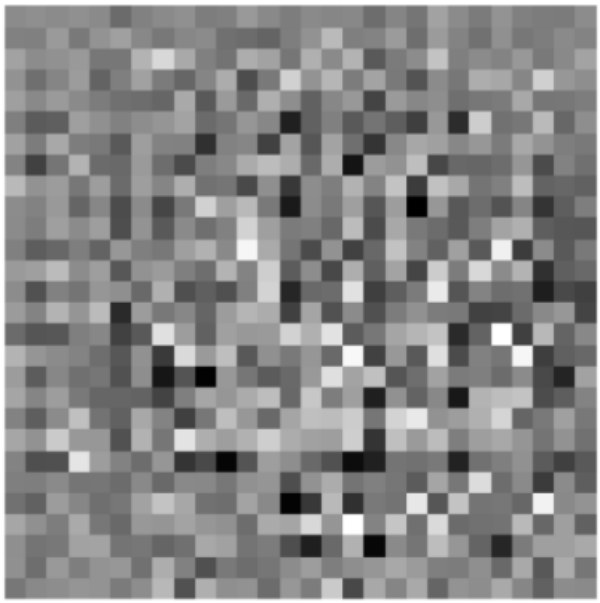
    model.add(layers.Conv2DTranspose(
        1, (5, 5), strides=(2, 2),
        padding="same", use_bias=False,
        activation="tanh"
    ))

    return model

generator = build_generator()
generator.summary()
```

[6]:
```
noise = tf.random.normal([1, latent_dim])
generated_image = generator(noise, training=False)

plt.imshow(generated_image[0, :, :, 0], cmap="gray")
plt.axis("off")
plt.show()
```



[7]:
```
def build_discriminator():
    model = keras.Sequential(name="discriminator")

    model.add(layers.Input(shape=(28, 28, 1)))

    model.add(layers.Conv2D(
        64, (5, 5), strides=(2, 2), padding="same"
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(
        128, (5, 5), strides=(2, 2), padding="same"
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation="sigmoid"))

    return model

discriminator = build_discriminator()
discriminator.summary()
```

[8]:
```
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(
        tf.ones_like(real_output), real_output
    )
    fake_loss = cross_entropy(
        tf.zeros_like(fake_output), fake_output
    )
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(
        tf.ones_like(fake_output), fake_output
    )
```

[9]:
```
generator_optimizer = keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = keras.optimizers.Adam(learning_rate=1e-4)
```

[10]:
```
train_dataset = (
    tf.data.Dataset
    .from_tensor_slices(x_train)
    .shuffle(60000)
    .batch(batch_size, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

print("Dataset ready")
```

```
Dataset ready
```

[12]:
```
def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    predictions = (predictions + 1.0) / 2.0

    fig = plt.figure(figsize=(4, 4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i, :, :, 0], cmap="gray")
        plt.axis("off")

    plt.suptitle(f"Epoch {epoch}")

    os.makedirs("images", exist_ok=True)

    plt.savefig(
        f"images/image_at_epoch_{epoch:04d}.png",
        bbox_inches="tight"
    )

    plt.show()
    plt.close()
```

[13]:
```
seed = tf.random.normal([
    num_examples_to_generate,
    latent_dim
])

print(seed.shape)
```

```
(16, 100)
```

[14]:
```
def train(dataset, epochs):
```

[11]:
```python
@tf.function
def train_step(images):
    noise = tf.random.normal([tf.shape(images)[0], latent_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gen_gradients = gen_tape.gradient(
        gen_loss, generator.trainable_variables
    )

    disc_gradients = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables
    )

    generator_optimizer.apply_gradients(
        zip(gen_gradients, generator.trainable_variables)
    )

    discriminator_optimizer.apply_gradients(
        zip(disc_gradients, discriminator.trainable_variables)
    )

    return gen_loss, disc_loss
```

[14]:
```python
def train(dataset, epochs):
    print("Beginning GAN Training")

    for epoch in range(epochs):
        gen_losses = []
        disc_losses = []

        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)

            gen_losses.append(float(gen_loss))
            disc_losses.append(float(disc_loss))

        avg_gen_loss = np.mean(gen_losses)
        avg_disc_loss = np.mean(disc_losses)

        print(
            f"Epoch {epoch + 1}/{epochs} - "
            f"Generator Loss: {avg_gen_loss:.4f}, "
            f"Discriminator Loss: {avg_disc_loss:.4f}"
        )

        if (epoch + 1) % 20 == 0:
            generate_and_save_images(
                generator,
                epoch + 1,
                seed
            )

    print("Training complete.")
```

[*]:
```python
train(train_dataset, EPOCHS)
```

```
Beginning GAN Training
Epoch 1/20 - Generator Loss: 0.8077, Discriminator Loss: 0.9907
Epoch 2/20 - Generator Loss: 0.7940, Discriminator Loss: 1.2919
Epoch 3/20 - Generator Loss: 0.8669, Discriminator Loss: 1.2270
```

```
[*]: train(train_dataset, EPOCHS)
```

```
Beginning GAN Training
Epoch 1/20 - Generator Loss: 0.8077, Discriminator Loss: 0.9907
Epoch 2/20 - Generator Loss: 0.7940, Discriminator Loss: 1.2919
Epoch 3/20 - Generator Loss: 0.8669, Discriminator Loss: 1.2270
Epoch 4/20 - Generator Loss: 0.8720, Discriminator Loss: 1.2025
Epoch 5/20 - Generator Loss: 0.8831, Discriminator Loss: 1.2931
```

```
[*]: generate_and_save_images(
    generator,
    EPOCHS,
    seed
)
```

```
[*]: noise = tf.random.normal([16, latent_dim])
generated_images = generator(noise, training=False)
generated_images = (generated_images + 1) / 2

plt.figure(figsize=(5, 5))

for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(generated_images[i, :, :, 0], cmap="gray")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
[*]: generator.save("generator.keras")
discriminator.save("discriminator.keras")

print("Models saved successfully")
```

RESULT:

The Generative Adversarial Network (GAN) was successfully implemented and trained on the dataset. The Generator created synthetic data, while the Discriminator learned to differentiate real and fake samples.

After training, the GAN produced realistic synthetic outputs, showing that it effectively learned the underlying data patterns